

A Reference Guide to Distributed Systems and Site Reliability Engineering

A working reference on failure modes, consistency models, and operational practice for production systems.

1. Foundations of Distributed Systems

1.1 What Makes a System Distributed

A distributed system is any collection of independent computers that appears to its users as a single coherent system. The defining property is not the number of machines involved but the fact that components communicate and coordinate by passing messages across a network rather than sharing memory directly. This single fact introduces almost every hard problem the field deals with: messages can be delayed, duplicated, reordered, or lost outright, and any two machines can disagree about what time it currently is.

Because failures in a distributed system are partial rather than total, a system can be simultaneously working correctly for some users and failing for others. This is fundamentally different from a single-machine program, where a crash typically halts everything at once. Distributed systems engineering is therefore less about preventing failure entirely and more about designing systems that continue operating usefully in the presence of failure.

1.2 The Eight Fallacies of Distributed Computing

Early in the history of networked computing, engineers at Sun Microsystems identified a set of assumptions that programmers habitually make when writing distributed software, all of which are false in practice. The network is not reliable. Latency is not zero. Bandwidth is not infinite. The network is not secure. Topology does not stay constant. There is not a single administrator. Transport cost is not zero. The network is not homogeneous.

These are widely referred to as the eight fallacies of distributed computing. Each one, when assumed rather than verified, tends to produce a specific class of production incident. Assuming zero latency produces systems that time out under real-world network conditions. Assuming infinite bandwidth produces systems that degrade badly under load. Assuming a reliable network produces retry logic that either does not exist or does not handle duplicate delivery correctly.

1.3 CAP Theorem

The CAP theorem states that a distributed data store can provide at most two of the following three guarantees simultaneously: Consistency, meaning every read receives the most recent write or an error; Availability, meaning every request receives a non-error response, without the guarantee that it contains the most recent write; and Partition Tolerance, meaning the system continues to operate despite an arbitrary number of messages being dropped or delayed between nodes.

In practice, because network partitions are a physical reality that cannot be engineered away, Partition Tolerance is not optional for any system that spans more than one machine. The real choice practitioners face is therefore between Consistency and Availability during a partition event. A system that chooses Consistency will refuse requests it cannot guarantee are correct, becoming unavailable during a partition. A system that chooses Availability will continue answering requests, accepting that

some of those answers may be stale.

Most production databases are not purely CP or purely AP in an absolute sense; they make this trade-off differently depending on the specific operation, and many offer tunable consistency levels that let an application choose per-query.

2. Consensus and Coordination

2.1 Why Consensus Is Hard

Consensus is the problem of getting a group of distributed nodes to agree on a single value, even when some nodes may fail or messages may be delayed. The FLP impossibility result, published in 1985 by Fischer, Lynch, and Paterson, proved that in an asynchronous network where even one node might fail, no algorithm can guarantee consensus will be reached in bounded time. This does not mean consensus is impossible in practice; it means any practical algorithm must make additional assumptions, typically about timing, in order to make progress.

2.2 Raft and Leader Election

Raft is a consensus algorithm designed explicitly for understandability, in contrast to the earlier Paxos family of algorithms, which are correct but notoriously difficult to reason about. Raft works by electing a single leader among a cluster of nodes. The leader is responsible for accepting all writes and replicating them to follower nodes. If the leader fails to send a heartbeat within a timeout window, followers assume it has crashed and hold a new election.

A critical detail is that a Raft cluster requires a majority of nodes, not merely a plurality, to agree before a write is considered committed. This is why Raft clusters are typically deployed with an odd number of nodes: a five-node cluster can tolerate the loss of two nodes and still form a majority of three, while a four-node cluster provides no additional fault tolerance over a three-node one, since it still only tolerates the loss of one node before losing majority.

2.3 Split-Brain Scenarios

A split-brain scenario occurs when a network partition causes two disjoint groups of nodes to each believe they are the legitimate leader or primary, and both continue accepting writes independently. This is one of the most dangerous failure modes in distributed systems because it can silently produce two divergent, conflicting versions of the same data, and reconciling them after the fact may require manual intervention or, in the worst case, data loss.

Quorum-based systems are designed specifically to prevent split-brain by requiring a strict majority before any group of nodes is permitted to act as the authoritative primary. A minority partition, by definition, cannot reach a majority of the total cluster and is therefore forced to become read-only or unavailable rather than risk diverging from the majority side.

3. Failure Modes in Production Systems

3.1 Connection Pool Exhaustion

A connection pool is a cache of established database connections maintained so that an application does not have to pay the cost of opening a new TCP connection and authenticating for every single query. Pools have a fixed maximum size. When every connection in the pool is simultaneously in use, any further request must wait for one to free up. If requests are arriving faster than connections are being released, the wait queue grows without bound, and requests begin timing out.

Connection pool exhaustion is frequently caused by a slow query holding a connection open far longer than expected, a connection leak where code fails to release a connection back to the pool after use, or a genuine traffic spike that exceeds the pool's configured capacity. From the outside, all three causes look identical: response times climb sharply, and eventually requests start failing outright. Diagnosing which of the three actually occurred requires correlating the timing of the slowdown against recent deployments, query logs, and traffic graphs together, since no single signal reliably distinguishes between them on its own.

3.2 Cascading Failures

A cascading failure begins when one component becomes slow or unavailable, and the resulting backpressure causes the components that depend on it to also become slow or unavailable, propagating outward through the system. A common trigger is a downstream service beginning to respond slowly rather than failing fast; callers hold open connections and threads waiting for a response that eventually arrives too late to be useful, exhausting their own resource limits in the process and becoming slow themselves.

Circuit breakers are the standard mitigation. A circuit breaker tracks the failure rate of calls to a dependency and, once that rate crosses a threshold, stops sending new requests to it entirely for a cooldown period, failing fast instead of waiting on a request that is likely to time out anyway. This protects the caller's own resources and, just as importantly, reduces load on the already-struggling downstream service, giving it room to recover.

3.3 Thundering Herd

A thundering herd occurs when a large number of clients simultaneously retry a failed request at the same moment, often because they all share the same retry interval or were all waiting on the same expired cache entry. The resulting spike can overwhelm a service that had otherwise nearly recovered, effectively re-triggering the original outage. Jittered backoff, where each client adds a small random delay to its retry interval, is the standard fix, since it spreads retries out over time instead of concentrating them at a single instant.

3.4 Deployment-Related Incidents

A disproportionate share of production incidents correlate closely in time with a recent deployment. This is not because deployments are inherently risky in some abstract sense, but because a deployment is the single most common way that new, untested code paths, configuration changes, or dependency version changes actually reach production traffic. For this reason, checking deployment history is almost always among the first steps in diagnosing an unexplained incident, and many incident response processes explicitly ask whether anything was deployed in the window immediately preceding the first symptom.

4. Observability

4.1 The Three Pillars

Observability is commonly described in terms of three data types: logs, metrics, and traces. Logs are discrete, timestamped records of individual events, useful for understanding exactly what happened at a specific point in time. Metrics are numeric measurements aggregated over time, such as request rate or error count, useful for spotting trends and setting alerting thresholds. Traces follow a single request as it moves through multiple services, useful for understanding where time is actually being spent in a system composed of many small services.

No single pillar is sufficient on its own. Metrics can tell you that response time increased, but not why. Logs can tell you that a specific error occurred, but not how common it was relative to normal traffic. Traces can show you which specific service call was slow, but not whether that slowness was typical or an anomaly. Effective incident diagnosis generally requires moving between all three.

4.2 Alert Fatigue

Alert fatigue occurs when a team receives so many alerts, including a high proportion of false positives or low-severity notifications, that they begin to habitually ignore or delay responding to all alerts, including genuinely critical ones. This is widely considered one of the most dangerous states an operations team can be in, since it defeats the entire purpose of having automated monitoring. The standard mitigation is to alert only on symptoms that are directly user-facing and actionable, rather than on every internal metric that deviates from its usual range.

4.3 Root Cause Correlation

Automated root-cause analysis tools generally work by correlating multiple independent signal types within a time window around a triggered alert, on the theory that a genuine incident will produce several related signals close together in time, while noise tends to be isolated. A single error log entry with no corresponding deployment, alert, or metric anomaly nearby is far more likely to be routine background noise than the signature of a real incident. Confidence in an automated diagnosis should scale with how many independent signal types actually corroborate each other, not with the raw volume of any single signal type alone.

5. Data Replication and Consistency

5.1 Synchronous vs Asynchronous Replication

In synchronous replication, a write is not acknowledged to the client until it has been confirmed by all (or a quorum of) replicas. This guarantees that an acknowledged write will survive the failure of any single replica, at the cost of higher write latency, since the client must wait for the slowest replica in the quorum to respond. In asynchronous replication, a write is acknowledged as soon as the primary has recorded it, and replicas catch up afterward. This is faster, but introduces a window during which a primary failure can lose writes that were acknowledged to the client but never made it to any replica.

5.2 Eventual Consistency

A system is eventually consistent if, given no new writes, all replicas will eventually converge to the same value, though at any given moment different replicas may temporarily disagree. This is a deliberately weaker guarantee than strong consistency, chosen because it allows a system to remain available and responsive even when replicas cannot immediately communicate. Many large-scale systems accept eventual consistency for data where a brief staleness window is tolerable, while reserving strong consistency for data where it is not, such as financial balances.

5.3 Read Repair and Anti-Entropy

Read repair is a technique where, if a read operation detects that replicas disagree, the system opportunistically writes the most recent value back to the stale replicas as a side effect of serving the read. Anti-entropy processes run independently and periodically, comparing replicas against each other in the background and reconciling differences even for data that has not been recently read. Together these mechanisms bound how long a replica can remain stale, without requiring every single write to pay the cost of synchronous replication.

6. Incident Response Practice

6.1 The First Ten Minutes

The initial minutes of an incident response are disproportionately valuable, and disproportionately prone to wasted effort. A common failure pattern is for the first responder to begin deep-diving into a single hypothesis before establishing basic situational awareness: what changed recently, which service is affected, and how many users are impacted. Establishing this context first, even briefly, tends to shorten total resolution time more than immediately pursuing the first plausible theory.

6.2 Postmortems

A postmortem is a structured written account of an incident, produced after resolution, covering the timeline of events, the root cause, the impact, and the concrete follow-up actions intended to prevent recurrence. The most effective postmortems are blameless: they focus on what conditions in the system allowed the incident to occur, rather than which individual made a mistake, on the grounds that a system which depends on no human ever making an error is not a resilient system.

A postmortem's value is realized only if the follow-up actions it identifies are actually completed. Teams that consistently write thorough postmortems but do not track follow-up items to completion tend to see the same categories of incident recur, since the underlying conditions were documented but never actually addressed.

6.3 Runbooks

A runbook is a documented, step-by-step procedure for responding to a specific, previously encountered type of incident. Runbooks are most valuable for incidents that are well understood but time-pressured, where looking up the correct response from memory under stress is more error-prone than following a written checklist. Runbooks that are not kept up to date as the underlying system changes become actively dangerous, since a responder following a stale runbook may take an action that was correct for an older version of the system but harmful for the current one.